

Báo cáo rà soát test code

<!-- framework: pytest | run_id: run_20260617_004149_236377 -->

Framework: pytest | Run ID: run_20260617_004149_236377

Tóm tắt kỹ thuật

- Framework: pytest
- Kỹ thuật kiểm thử: Hybrid (kết hợp nhiều kỹ thuật)
- Kết quả chạy: Không có bằng chứng thực thi nào được cung cấp; do thiếu source code và execution evidence nên các test không thể chạy được.
- Vấn đề lớn nhất: Các test import các hàm apply_store_credit, calculate_discount, final_total, shipping_fee từ module source_under_test nhưng không có file nguồn nào được đưa ra, dẫn tới lỗi ImportError ngay khi thu thập test.
- Tài liệu được xem xét: Test code pytest_review_sample.py (được cung cấp). Không có source code, test plan, hoặc requirement chi tiết.

Lỗi nghiêm trọng

- ImportError do thiếu source code
 - Bằng chứng: Test code có dòng from source_under_test import apply_store_credit, calculate_discount, final_total, shipping_fee nhưng trong prompt không có bất kỳ đoạn [SOURCE CODE] nào.
 - Ảnh hưởng: Các test sẽ không thể được thu thập và chạy, gây lỗi ngay từ đầu và làm cho toàn bộ bộ kiểm thử vô dụng.
 - Cách sửa: Cung cấp file source_under_test.py (hoặc module tương ứng) chứa các hàm được import, hoặc chỉnh sửa import để trở tới đúng module hiện có.
-
- Thiếu thông tin yêu cầu và kế hoạch kiểm thử
 - Bằng chứng: Các phần "Requirement JSON" và "Test Plan JSON" đều ghi "Needs clarification".
 - Ảnh hưởng: Không thể xác định xem các giá trị kỳ vọng (ví dụ calculate_discount(1000, "VIP") == 50.0) có đúng hay không, dẫn tới khả năng các assertion là sai lệch.
 - Cách sửa: Cung cấp yêu cầu chi tiết (định nghĩa quy tắc giảm giá, phí vận chuyển, giới hạn credit, v.v.) và kế hoạch kiểm thử để xác nhận tính đúng đắn của các giá trị mong đợi.

Test còn thiếu

- Không thể đánh giá vì không có yêu cầu/plan để so sánh.
- Gợi ý: Khi yêu cầu được cung cấp, cần bổ sung các test:
- Trường hợp giảm giá cho khách hàng không phải VIP.
- Kiểm tra phí vận chuyển cho đơn hàng nhẹ, quốc tế, không fragile.
- Kiểm tra apply_store_credit khi credit lớn hơn subtotal (đã có nhưng cần xác nhận logic).
- Kiểm tra final_total với các giá trị biên (subtotal = 0, weight = 0, destination không hợp lệ, v.v.).

Assertion yếu

- Các assertion hiện tại đều so sánh giá trị cố định, nhưng không có bằng chứng rằng các giá trị này đúng theo yêu cầu.
- Ví dụ: `assert calculate_discount(1000, "VIP") == 50.0` - nếu quy tắc giảm giá là 5% thì giá trị đúng là 50, nhưng nếu quy tắc thay đổi thì test sẽ sai mà không có tài liệu để kiểm chứng.
- Cách sửa: Thêm comment hoặc lấy giá trị kỳ vọng từ tài liệu yêu cầu; hoặc tính giá trị kỳ vọng trong test dựa trên hàm helper để tránh hard-code.

Rủi ro maintainability

- Phụ thuộc vào module không tồn tại → gây lỗi khi dự án thay đổi cấu trúc thư mục.
- Hard-code giá trị kỳ vọng mà không liên kết với yêu cầu → khi yêu cầu thay đổi, các test sẽ cần cập nhật thủ công, dễ bỏ sót.
- Không có mock hoặc fixture để cô lập các hàm phụ thuộc vào môi trường (nếu có).

Gợi ý sửa ngay

1. Cung cấp source code `source_under_test.py` chứa các hàm được import, hoặc chỉnh sửa import cho đúng.
2. Thêm tài liệu yêu cầu (Requirement JSON) và kế hoạch kiểm thử (Test Plan JSON) để các giá trị kỳ vọng có cơ sở.
3. Biến các giá trị kỳ vọng thành biến dựa trên yêu cầu (ví dụ: `discount = subtotal * rate_from_requirements`) thay vì hard-code.
4. Thêm test cho các trường hợp biên (negative, zero, giá trị không hợp lệ) dựa trên yêu cầu.
5. Kiểm tra và xử lý `ImportError` trong file test bằng cách sử dụng `try/except` để đưa ra thông báo rõ ràng nếu module không tồn tại.

Nếu không cung cấp source code và yêu cầu chi tiết, không thể tiến hành đánh giá sâu hơn và các test hiện tại sẽ luôn bị lỗi.